

House Price Prediction with Advanced Feature Engineering, Ensemble Modeling & Explainable AI

1. Project Overview

Project Title

House Price Prediction with Advanced Feature Engineering, Ensemble Modeling & Explainable AI

Problem Statement

The goal of this project is to predict the sale price of a house based on its characteristics such as:

- Overall quality
- Living area
- Basement area
- Garage capacity
- Year built
- Neighborhood
- Number of bathrooms
- Lot area
- Other property characteristics

This is a **supervised machine learning regression problem** because:

- We have historical houses.
- Each house has input features.
- Each training house has a known **SalePrice**.
- We learn the relationship between the features and **SalePrice**.
- For a new house, the model predicts its price.

Dataset

We used the Kaggle **House Prices: Advanced Regression Techniques** dataset.

Training data:

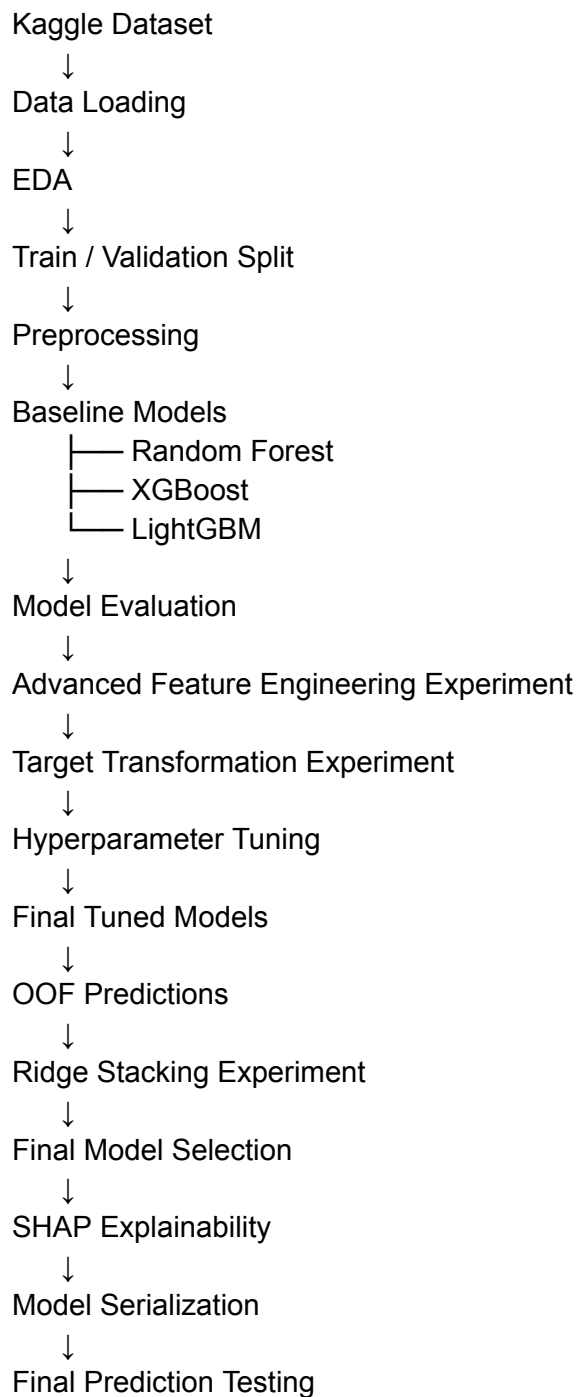
- 1460 houses
- 81 columns
- 80 input features

- 1 target feature: **SalePrice**

SalePrice is the value we want to predict.

2. Complete Project Pipeline

The complete pipeline was:



We intentionally did **not** deploy the project online.

3. Step 1 — Load the Dataset

We loaded:

```
train = pd.read_csv("train.csv")
```

Why?

We need to bring the CSV data into Python/Pandas.

What happens if we skip this?

There is no data available for analysis or training.

4. Step 2 — Understand the Dataset

We checked:

```
train.shape  
train.columns  
train.dtypes  
train.info()  
train.isnull().sum()  
train.duplicated().sum()
```

We discovered:

```
Rows = 1460  
Columns = 81  
Numerical columns = 38  
Categorical columns = 43  
Target = SalePrice
```

After removing **SalePrice**:

Input features = 80

Why did we inspect these things?

Because before training a model, we need to understand:

- What types of variables exist?
- Which columns are numerical?
- Which columns are categorical?
- Are values missing?
- Are there duplicate records?
- Is the target distribution normal or skewed?

What if we skipped EDA?

We might:

- Handle missing values incorrectly.
- Treat categorical data incorrectly.
- Miss outliers.
- Miss target skewness.
- Introduce data leakage.
- Build a model without understanding the data.

5. Step 3 — Missing Value Analysis

We found 19 columns containing missing values.

Important examples:

PoolQC
MiscFeature
Alley
Fence
MasVnrType
FireplaceQu
GarageType
GarageYrBlt
GarageFinish
GarageQual
GarageCond
BsmtQual
...

A very important observation was:

Missing does not always mean bad data or unknown data.

For example:

PoolQC = NaN

often means:

The house does not have a pool.

Similarly:

GarageType = NaN

usually means:

The house does not have a garage.

Why was this important?

Because replacing every missing value with the average/mode can destroy the meaning of the data.

What if we blindly filled everything with the mean?

For categorical variables, mean doesn't even make sense.

If we used the most frequent category everywhere, we could incorrectly tell the model that a house has a common feature when actually it does not have that feature.

6. Step 4 — Duplicate Check

We used:

```
train.duplicated().sum()
```

Result:

0

Meaning

There were no exact duplicate rows.

Why check duplicates?

Duplicates can cause certain observations to receive too much influence during training.

7. Step 5 — Target Analysis

Our target is:

SalePrice

Statistics:

Mean $\approx$ \$180,921

Median = \$163,000

Minimum = \$34,900

Maximum = \$755,000

We plotted the histogram of `SalePrice`.

We discovered right skewness.

Definition: Right-skewed distribution

A right-skewed distribution has:

- Most observations toward the lower/middle side.
- A long tail toward high values.

In our dataset:

Many houses $\rightarrow$ 100K –250K

Few houses $\rightarrow$ 400K –750K

That creates the long right tail.

Also:

Mean > Median

which supports the presence of right skew.

Why does skewness matter?

Very expensive houses can have a disproportionate impact on regression loss.

So we later tested a log target transformation.

8. Step 6 — Correlation Analysis

We calculated correlations with `SalePrice`.

Top relationships included:

OverallQual ≈ 0.79
GrLivArea ≈ 0.71
GarageCars ≈ 0.64
GarageArea ≈ 0.62
TotalBsmtSF ≈ 0.61
1stFlrSF ≈ 0.61
FullBath ≈ 0.56
TotRmsAbvGrd ≈ 0.53
YearBuilt ≈ 0.52

What is correlation?

Correlation measures the strength and direction of a **linear relationship**.

Range:

+1 $\rightarrow$ strong positive
0 $\rightarrow$ no linear relationship
-1 $\rightarrow$ strong negative

Important interview point

Correlation does not mean causation.

Also:

Low correlation does not automatically mean the feature is useless.

Tree models can capture nonlinear relationships.

9. Important Feature: OverallQual

We discovered that **OverallQual** was one of the strongest features.

It is a rating from 1–10 representing overall material and finish quality.

Average prices increased substantially as quality increased.

Example:

Quality 4 → average price ≈ \$108K

Quality 6 → average price ≈ \$162K

Quality 8 → average price ≈ \$275K

Quality 10 → average price ≈ \$439K

Interview insight

This provides domain intuition:

House quality is strongly associated with selling price.

10. Important Feature: GrLivArea

GrLivArea means:

Above-ground living area.

We found a strong positive relationship between **GrLivArea** and **SalePrice**.

We also investigated extreme values.

Examples of very large living areas included:

5642 sq ft → \$160K

4676 sq ft → \$184.75K

4476 sq ft → \$745K

Only 4 houses had:

GrLivArea > 4000

Why did we investigate this?

These observations could behave as outliers and strongly influence regression models.

We did not blindly remove them because an unusual value is not automatically an incorrect value.

11. Step 7 — Separate X and y

We created:

```
X = train.drop("SalePrice", axis=1)
y = train["SalePrice"]
```

Therefore:

X = input features
y = target

Shapes:

```
X = (1460, 80)
y = (1460,)
```

Why?

Every supervised ML problem needs:

X → inputs
y → answers

12. Step 8 — Train / Validation Split

We used:

```
train_test_split(
```

```
X,  
y,  
test_size=0.20,  
random_state=42  
)
```

Result:

Training = 1168 houses
Validation = 292 houses

So this is an:

80/20 split.

Why?

We need data the model did not learn from to evaluate generalization.

Why `random_state=42`?

It makes the split reproducible.

If we run the code again, the same rows go into the same sets.

Why not train on all 1460 immediately?

Because then we have no independent holdout data for measuring how the model behaves on unseen houses.

13. Step 9 — Preprocessing

We had two types of features:

Numerical = 37 input features
Categorical = 43 input features

Numerical preprocessing

We used:

```
SimpleImputer(strategy="median")
```

Definition: Median imputation

Missing numerical values are replaced by the median of the training column.

Why median?

Median is less sensitive to extreme values than the mean.

Example:

10, 11, 12, 13, 1000

Mean is heavily affected by 1000.

Median is still 12.

Categorical preprocessing

We used:

```
SimpleImputer(...)  
OneHotEncoder(handle_unknown="ignore")
```

Definition: One-Hot Encoding

It converts a categorical feature into binary columns.

Example:

Neighborhood:

RL
RM
FV

becomes something like:

Neighborhood_RL
Neighborhood_RM
Neighborhood_FV

with 0/1 values.

Why `handle_unknown="ignore"`?

If a future house contains a category that wasn't present during training, the encoder does not crash.

This is important in real prediction systems.

14. Data Leakage

This is one of the most important interview concepts in the project.

Definition: Data leakage

Data leakage occurs when information from outside the training data improperly influences model training.

Our safe sequence was:

Train data



Fit preprocessing



Transform train

Validation data



Only transform using learned rules

We did NOT:

Entire dataset



fit preprocessing



split

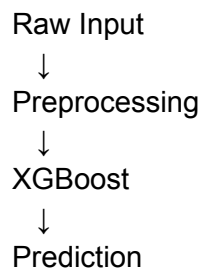
Why?

If validation information influences preprocessing, validation performance becomes overly optimistic.

15. Why preprocessing was stored in a Pipeline

We used Scikit-learn `Pipeline`.

Conceptually:



Why is this important?

It guarantees the same preprocessing is applied during inference.

It also makes model saving and deployment much easier.

16. Baseline Models

We tested three models:

1. Random Forest
2. XGBoost
3. LightGBM

Why multiple models?

Because different algorithms learn patterns differently.

We wanted to compare them empirically rather than assuming one was best.

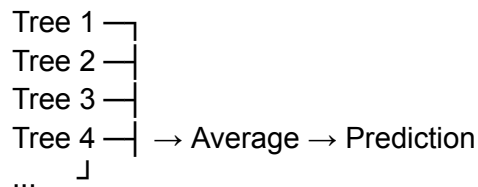
17. Random Forest

Definition

Random Forest is an ensemble of decision trees using **bagging**.

Multiple trees are trained using different samples/features and their predictions are combined, usually by averaging in regression.

Conceptually:



Bagging definition

Bagging = **Bootstrap Aggregating**.

Multiple models are trained independently and their predictions are aggregated.

Why did we use Random Forest?

It is:

- Strong on tabular data.
- Relatively robust.
- Good at nonlinear relationships.
- Less sensitive to feature scaling.
- A useful baseline ensemble model.

What if we didn't use Random Forest?

We would lose a useful comparison against boosting methods.

It also gives us a model with a different ensemble philosophy.

18. Random Forest Baseline Hyperparameters

We initially used:

`n_estimators = 200`

```
random_state = 42  
n_jobs = -1
```

n_estimators

Number of trees.

More trees generally make predictions more stable but increase computation.

random_state

Controls reproducibility.

n_jobs=-1

Uses all available CPU cores.

19. Tuned Random Forest

Our tuning found:

```
n_estimators = 600  
max_depth = 15  
min_samples_split = 2  
min_samples_leaf = 1  
max_features = sqrt
```

max_depth

Maximum depth of each tree.

Higher depth:

- More complex patterns.
- Higher overfitting risk.

Lower depth:

- Simpler model.
- May underfit.

min_samples_split

Minimum number of samples needed to split a node.

Higher values make trees less complex.

min_samples_leaf

Minimum number of observations required in a leaf.

Higher values smooth the model and reduce overfitting.

max_features

Number of features considered when splitting.

Using **sqrt** increases randomness and diversity among trees.

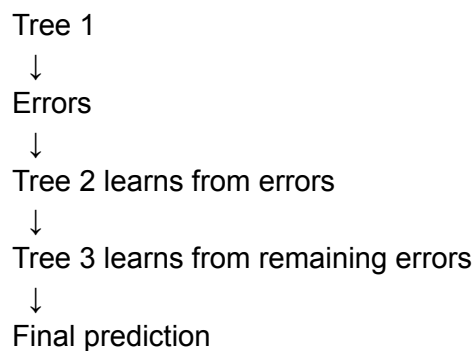
20. XGBoost

Definition

XGBoost is an optimized implementation of **gradient boosting decision trees**.

Unlike Random Forest, trees are built **sequentially**.

Conceptually:



Gradient Boosting definition

Gradient boosting builds models sequentially, with each new model trying to reduce the previous ensemble's prediction error using gradient-based optimization.

Why XGBoost?

XGBoost is often extremely strong on structured/tabular data because it can capture:

- Nonlinear relationships.
- Feature interactions.
- Complex patterns.
- Different scales between numerical variables.

Why did XGBoost become our final model?

Because after tuning:

$R^2 \approx 0.919$

RMSE $\approx$ \$24,870

MAE $\approx$ \$15,719

This was our best overall validation result.

21. XGBoost Parameters

Our initial XGBoost used:

```
n_estimators = 500
learning_rate = 0.03
max_depth = 4
random_state = 42
n_jobs = -1
```

Our tuned final XGBoost used:

```
n_estimators = 900
learning_rate = 0.02
max_depth = 3
min_child_weight = 3
subsample = 0.9
colsample_bytree = 0.8
reg_alpha = 0.01
reg_lambda = 1
```

Definitions

n_estimators

Number of boosting trees.

More trees allow the model to learn more gradually, especially with a small learning rate.

learning_rate

Controls how much each new tree contributes.

Lower learning rate:

- Slower learning.
- Often requires more trees.
- Can improve generalization.

Our tuned value was:

0.02

max_depth

Maximum depth of each tree.

Our final value:

3

A shallow tree reduces complexity and overfitting.

min_child_weight

Controls the minimum amount of weight/sample information needed for further splitting.

Larger values make the model more conservative.

subsample

Fraction of training rows used by each boosting round.

We used:

0.9

So each tree uses about 90% of the available training rows.

This introduces randomness and can reduce overfitting.

colsample_bytree

Fraction of features sampled for each tree.

We used:

0.8

This means each tree sees roughly 80% of the features.

reg_alpha

L1 regularization.

We used:

0.01

It helps control model complexity and can push some feature effects toward zero.

reg_lambda

L2 regularization.

We used:

1

It penalizes large model weights and helps reduce overfitting.

22. Why tune hyperparameters?

Definition

Hyperparameters are settings chosen before model training.

Examples:

learning_rate
max_depth
n_estimators
subsample

They are different from model parameters, which the algorithm learns during training.

Why tune them?

Default parameters are not guaranteed to be optimal for a particular dataset.

What if we didn't tune?

We might leave performance on the table or get unnecessary overfitting.

23. Hyperparameter Tuning Method

We used:

RandomizedSearchCV

with:

12 parameter combinations
5-fold CV

which resulted in:

60 fits

Definition: RandomizedSearchCV

Instead of testing every possible combination, it randomly samples combinations from defined parameter distributions.

Why RandomizedSearchCV instead of GridSearchCV?

GridSearch checks every combination.

If there are many parameters, the number of combinations grows rapidly.

Randomized search lets us explore a larger space with fewer experiments.

24. Cross-Validation

We used 5-fold cross-validation during tuning.

Definition

5-fold cross-validation divides the training data into 5 parts.

Fold 1 → validation

Fold 2 → validation

Fold 3 → validation

Fold 4 → validation

Fold 5 → validation

Each observation is used as validation once.

Why?

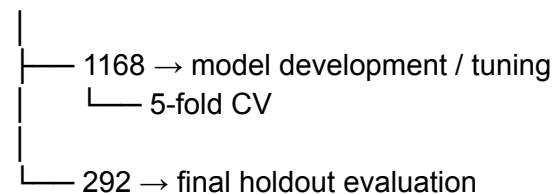
It provides a more reliable estimate for model selection than a single internal split.

Important

The 292-house holdout was kept outside tuning.

So:

1460 houses



25. LightGBM

Definition

LightGBM is another gradient boosting framework optimized for speed and efficiency.

It uses histogram-based learning and grows trees efficiently.

Why did we use LightGBM?

Because:

- It is strong on tabular data.
- It can be faster than many boosting implementations.
- It provides diversity compared with Random Forest and XGBoost.
- Diverse models can be useful for stacking.

What if we didn't use LightGBM?

We would have one less boosting model and less diversity for the ensemble experiment.

26. LightGBM Tuned Parameters

Our tuned LightGBM found:

```
n_estimators = 700
learning_rate = 0.08
max_depth = 3
num_leaves = 31
min_child_samples = 20
subsample = 0.9
colsample_bytree = 0.7
reg_alpha = 0.1
reg_lambda = 1
```

Definitions

`n_estimators` = number of trees.

`learning_rate` = contribution of each tree.

`max_depth` = maximum tree depth.

`num_leaves` = maximum number of leaves in each tree.

More leaves usually increase model complexity.

`min_child_samples` = minimum number of samples in a leaf.

`subsample` = percentage of training rows used.

`colsample_bytree` = percentage of features used per tree.

`reg_alpha` = L1 regularization.

`reg_lambda` = L2 regularization.

27. Model Comparison

Our tuned results:

Random Forest
MAE $\approx$ 18,224
RMSE $\approx$ 32,565
 $R^2 \approx$ 0.862

XGBoost
MAE $\approx$ 15,719
RMSE $\approx$ 24,870
 $R^2 \approx$ 0.919

LightGBM
MAE $\approx$ 17,073
RMSE $\approx$ 28,265
 $R^2 \approx$ 0.896

Therefore:

XGBoost was the strongest individual model.

28. Advanced Feature Engineering

We experimented with 15 engineered features.

Examples:

TotalSF
TotalBsmtFin
TotalBathrooms
TotalPorchSF
HouseAge
RemodelAge
GarageAge
TotalLivingArea
OverallQual_GrLivArea
OverallQual_TotalSF

HasGarage
HasBsmt
HasFireplace
HasPool
Has2ndFloor

Examples

TotalSF

$\text{TotalBsmtSF} + \text{1stFlrSF} + \text{2ndFlrSF}$

TotalBathrooms

FullBath
+ $0.5 \times \text{HalfBath}$
+ BsmtFullBath
+ $0.5 \times \text{BsmtHalfBath}$

HouseAge

$\text{YrSold} - \text{YearBuilt}$

RemodelAge

$\text{YrSold} - \text{YearRemodAdd}$

HasGarage

1 if GarageArea > 0
0 otherwise

Interaction features

Example:

$\text{OverallQual} \times \text{GrLivArea}$

This captures interactions between quality and size.

29. Why Feature Engineering?

Definition

Feature engineering means transforming raw variables into more useful representations for machine learning.

Why?

A raw feature may not fully express the concept we want.

For example:

YearBuilt
YrSold

separately tell the model when the house was built and sold.

But:

HouseAge = YrSold - YearBuilt

directly tells the model the age at sale.

What if we didn't do feature engineering?

The model can still perform well, especially XGBoost, but it may have to learn relationships that we could represent explicitly.

Important result

Feature engineering did **not** improve our validation score.

Our original XGBoost:

$R^2 \approx 0.916$

XGBoost + feature engineering:

$R^2 \approx 0.910$

Therefore, we did not use the feature-engineered pipeline as the final production pipeline.

Interview answer

"I experimented with domain-driven feature engineering, but I selected the final model based on validation performance rather than assuming engineered features would always help."

This is a strong answer.

30. Target Transformation

We tested:

```
np.log1p(SalePrice)
```

Why?

Because **SalePrice** was right-skewed.

Definition: Log transformation

A logarithmic transformation compresses large values and can reduce skewness.

Result

Original XGBoost + FE:

$R^2 \approx 0.910$

Log-target version:

$R^2 \approx 0.908$

It performed slightly worse.

Decision

We did not use log target in the final model.

Interview lesson

"I don't assume a theoretically reasonable transformation will improve performance; I validate it experimentally."

31. Model Evaluation Metrics

We used:

1. MAE
 2. RMSE
 3. R^2
-

32. MAE

Definition

Mean Absolute Error:

$MAE = \text{average}(|\text{actual} - \text{predicted}|)$

Meaning

It tells us how far predictions are from actual prices on average.

Our final MAE:

$\approx \$15,718$

Interpretation

The average absolute prediction error was about \$15.7K.

Advantage

Easy to interpret.

Limitation

It treats all errors linearly.

33. RMSE

Definition

Root Mean Squared Error:

$$\text{RMSE} = \sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$$

Why use RMSE?

It penalizes large errors more strongly.

Our final RMSE

≈ \$24,870

Why was RMSE larger than MAE?

Because some predictions had relatively large errors, and squaring magnifies them.

Interview answer

"MAE tells me the average absolute error, while RMSE penalizes large errors more heavily."

34. R^2

Definition

R^2 measures the proportion of variance in the target explained by the model relative to a mean-based baseline.

Formula:

$$R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$$

Our result:

$$R^2 \approx 0.919$$

Meaning

Approximately 91.9% of the variation in house prices was explained by the model on our held-out validation set.

Important

Do NOT call this:

91.9% accuracy.

For regression, R^2 is not classification accuracy.

35. Why We Should Not Say "90.82% Accuracy"

We calculated:

Mean Error % $\approx$ 9.18%

Average Closeness $\approx$ 90.82%

for the validation set.

But this is our own derived "closeness" calculation.

It is not the standard ML accuracy metric.

Correct interview reporting

Say:

"The tuned XGBoost achieved R^2 of 0.919, RMSE of approximately \$24.9K, and MAE of approximately \$15.7K on the held-out validation set."

Do not say:

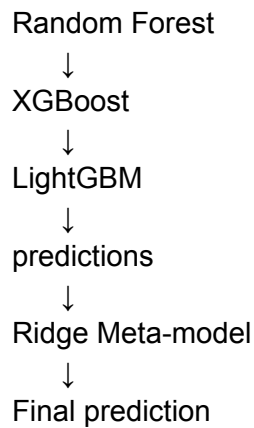
"The model has 90.82% accuracy."

36. Model Stacking

Definition

Stacking is an ensemble technique where multiple base models make predictions, and another model called a **meta-model** learns how to combine those predictions.

Our architecture:



Important

Stacking does NOT choose the single best model.

It uses predictions from **all base models**.

37. Why Stacking?

Different models can have different error patterns.

Example:

Random Forest → good on some houses

XGBoost → good on others

LightGBM → good on others

A meta-model can learn how to combine those predictions.

What if we didn't use stacking?

We would simply choose one model.

That is model selection, not stacking.

38. OOF Predictions

Definition

OOF = **Out-of-Fold prediction**.

A prediction made for an observation by a model that **did not train on that observation**.

Why?

Because using in-sample predictions for stacking causes leakage.

Example of bad approach:

Train XGBoost on House A



Predict House A



Give prediction to meta-model

The model has already seen House A.

That prediction is overly optimistic.

Proper approach

5-fold CV:

Fold 1 → train on other folds → predict Fold 1

Fold 2 → train on other folds → predict Fold 2

...

Fold 5 → train on other folds → predict Fold 5

Every training house receives an honest prediction.

39. Why 5 Folds?

We used 5 folds because it gives a practical balance between:

- reliable validation
- computational cost

It is not a magic number.

3-fold or 10-fold are also possible depending on data size and computation.

40. Ridge Regression Meta-model

Definition

Ridge Regression is linear regression with **L2 regularization**.

Formula conceptually:

Loss = prediction error + $\alpha \times \text{sum}(\text{coefficients}^2)$

We used:

alpha = 1.0

Why Ridge as meta-model?

The predictions from RF, XGBoost, and LightGBM are likely correlated.

Ridge handles correlated predictors better than ordinary linear regression because regularization stabilizes the coefficients.

What if we used ordinary linear regression?

The meta-model could become more sensitive to multicollinearity.

What if we used a very complex meta-model?

It might overfit the OOF predictions.

So a simple regularized meta-model is a sensible choice.

41. Stacking Results

Our final stacking experiment produced approximately:

Tuned XGBoost

$R^2 \approx 0.919$
RMSE $\approx 24,870$

Final Stacking
 $R^2 \approx 0.910$
RMSE $\approx 26,318$

Therefore:

Stacking did not beat tuned XGBoost.

Final decision

We selected:

Tuned XGBoost

not stacking.

Why?

Because model selection should be based on validation performance.

42. Final Model

Our final model is:

Tuned XGBoost

Final hyperparameters:

n_estimators = 900
learning_rate = 0.02
max_depth = 3
min_child_weight = 3
subsample = 0.9
colsample_bytree = 0.8
reg_alpha = 0.01
reg_lambda = 1

Validation performance:

MAE $\approx \$15,718$
RMSE $\approx \$24,870$

$R^2 \approx 0.919$

43. SHAP

Definition

SHAP = **SH**apley **ADD**itive **exPLAN**ations.

It is based on Shapley values from cooperative game theory.

SHAP explains how each feature contributes to a model prediction.

Why SHAP?

Tree-based models are powerful but can be difficult to interpret.

SHAP gives us:

- Global importance.
 - Direction of impact.
 - Individual prediction explanations.
-

44. SHAP Global Explanation

Our most important features included:

OverallQual
GrLivArea
TotalBsmntSF
BsmntFinSF1
GarageCars
YearBuilt
YearRemodAdd
LotArea

Interview interpretation

Higher:

- OverallQual

- GrLivArea
- TotalBsmntSF
- GarageCars

generally push predicted price upward.

45. SHAP Summary Plot

The SHAP beeswarm plot shows:

- feature importance
- direction
- high/low feature values

Color interpretation:

Red = high feature value

Blue = low feature value

Horizontal position:

Right → increases prediction

Left → decreases prediction

46. SHAP Waterfall Plot

A waterfall plot explains one house.

It answers:

"Why did the model predict this specific price?"

Example concepts:

OverallQual → pushes price up/down

GrLivArea → pushes price up/down

GarageCars → pushes price up/down

HouseAge → pushes price up/down

What if we didn't use SHAP?

The model would still predict correctly, but we would lose interpretability.

It would become harder to explain:

Why did the model predict this price?

47. Model Serialization

We saved:

```
final_house_price_model.joblib  
final_xgboost_model.joblib  
preprocessor.joblib  
final_metrics.joblib
```

Definition: Model serialization

Saving a trained model to disk so it can be reused later without retraining.

We used:

```
joblib.dump(...)
```

and later:

```
joblib.load(...)
```

Why?

Training can be expensive.

We don't want to retrain every time we make a prediction.

48. Production Pipeline

Our main saved pipeline contains:

Raw input



Preprocessing
↓
Tuned XGBoost
↓
Prediction

Why save preprocessing with the model?

Because prediction data must go through the exact same preprocessing used during training.

If training uses one-hot encoding and median imputation but inference uses something different, predictions can become invalid.

49. Prediction on a New House

At inference time, we don't need the huge training code.

Conceptually:

```
model = joblib.load(
    "final_house_price_model.joblib"
)

prediction = model.predict(
    new_house
)
```

This is why we separate:

Training

EDA
preprocessing
training
tuning
evaluation

from:

Inference

load model

input house
predict

50. Validation Example

One validation house gave:

Actual = \$154,500
Predicted = \$147,568.78
Error = \$6,931.22
Error $\approx$ 4.49%

Another:

Actual = \$163,500
Predicted = \$174,139.45
Error = \$10,639.47
Error $\approx$ 6.51%

These examples are useful for demonstration, but **two houses are not enough to evaluate the model**.

The overall 292-house validation result is more meaningful.

51. Kaggle Test Set

The Kaggle `test.csv` contains input features but **does not contain SalePrice**.

Therefore:

Training set
→ actual prices available

Validation set
→ actual prices available

Kaggle test set
→ prices hidden

We can predict Kaggle test houses, but cannot locally calculate their true accuracy without the hidden labels.

52. Why We Didn't Deploy

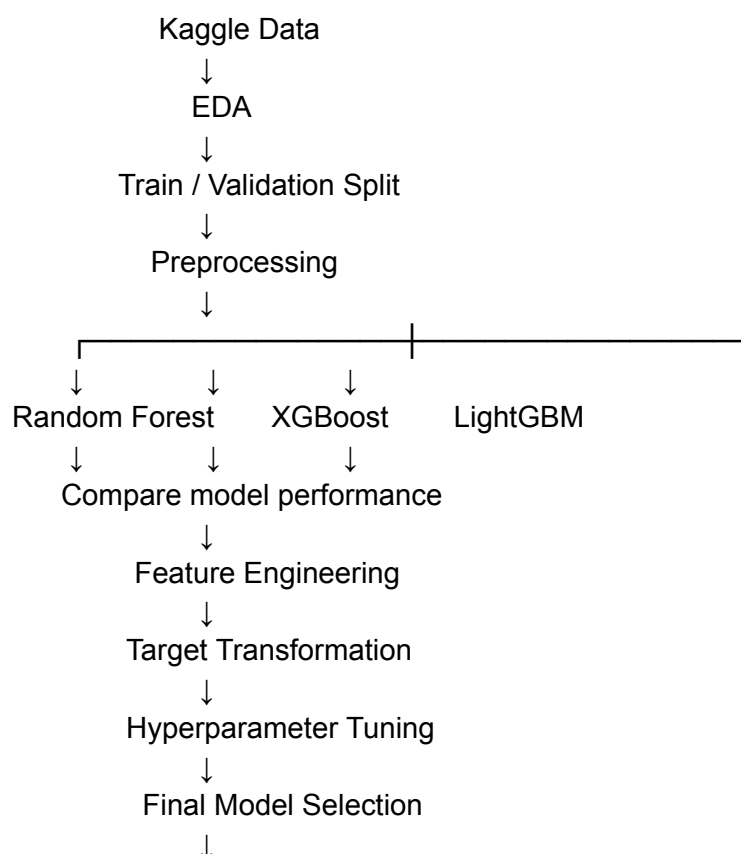
We intentionally skipped online deployment.

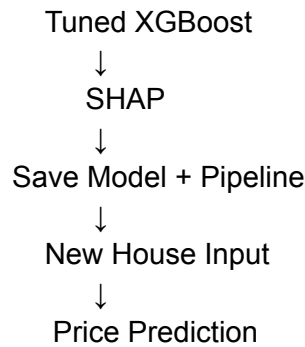
The project therefore focuses on:

- ML modeling
- Feature engineering
- Ensemble methods
- Hyperparameter tuning
- Explainability
- Reproducible model saving

A Streamlit app can be added later if needed.

53. Final Project Architecture





54. Most Important Interview Questions

Q1. What type of ML problem is this?

Answer:

It is a supervised regression problem because we have labeled historical data and the target variable, SalePrice, is continuous.

Q2. Why did you choose XGBoost?

Answer:

XGBoost performs very well on structured tabular data and captures nonlinear relationships and feature interactions. In my experiments, it achieved the best validation performance after hyperparameter tuning.

Q3. Why did you use Random Forest?

Answer:

Random Forest provided a strong bagging-based baseline and gave us a model with a different ensemble strategy from boosting models.

Q4. Random Forest vs XGBoost?

Answer:

Random Forest uses bagging and trains trees relatively independently, while XGBoost uses boosting and builds trees sequentially to correct previous errors.

Q5. XGBoost vs LightGBM?

Answer:

Both are gradient boosting methods, but LightGBM is optimized heavily for computational efficiency and uses histogram-based learning and leaf-oriented tree growth. XGBoost is also highly optimized and has a strong reputation for tabular problems. I compared both empirically rather than assuming one was better.

Q6. Why did you use three models?

Answer:

I wanted to compare different ensemble strategies and also test whether their different error patterns could benefit a stacking ensemble.

Q7. What is stacking?

Answer:

Stacking is an ensemble method where multiple base models make predictions and a meta-model learns how to combine those predictions.

Q8. Does stacking choose the best model?

Answer:

No. Stacking uses predictions from all base models. The meta-model learns how to combine them. Choosing only the best model is model selection, not stacking.

Q9. Why use OOF predictions?

Answer:

OOF predictions ensure that the meta-model receives predictions from models that did not train on those observations, preventing information leakage.

Q10. What is data leakage?

Answer:

Data leakage occurs when information that should be unavailable during training influences the model. For example, fitting preprocessing on the entire dataset before splitting can leak validation information into training.

Q11. Why do preprocessing after splitting?

Answer:

Because imputation and encoding statistics should be learned only from the training data. The validation set should only be transformed using those learned rules.

Q12. Why did you use One-Hot Encoding?

Answer:

Our dataset contains categorical features such as Neighborhood and KitchenQual. One-Hot Encoding converts them into numerical binary features that machine learning models can use.

Q13. Why `handle_unknown="ignore"`?

Answer:

It prevents prediction failures if a new category appears at inference time that was not seen during training.

Q14. Why did you use median imputation?

Answer:

Median imputation is robust to outliers and is a simple, stable approach for missing numerical values.

Q15. What does `n_estimators` mean?

Answer:

It is the number of trees or boosting rounds used by the ensemble.

Q16. What does `learning_rate` mean?

Answer:

It controls how strongly each new boosting tree contributes to the final prediction.

Q17. Why use a low learning rate?

Answer:

A lower learning rate makes learning more gradual and can improve generalization when combined with enough trees.

Q18. What does `max_depth` do?

Answer:

It limits the depth of each tree and therefore controls model complexity.

Q19. What is regularization?

Answer:

Regularization adds a penalty for model complexity to reduce overfitting.

Q20. Difference between L1 and L2 regularization?

Answer:

L1 uses the absolute value of coefficients and can encourage sparsity, while L2 uses squared coefficients and smoothly shrinks them.

In our XGBoost model:

reg_alpha → L1

reg_lambda → L2

Q21. Why use **subsample**?

Answer:

It controls what fraction of training rows are used by each boosting round. Using less than 100% introduces randomness and can reduce overfitting.

Q22. Why use **colsample_bytree**?

Answer:

It controls what fraction of features are considered for each tree and adds randomness, which can improve generalization.

Q23. What is **min_child_weight**?

Answer:

It controls how conservative XGBoost is when creating child nodes. Larger values require more information before allowing a split.

Q24. Why use RandomizedSearchCV?

Answer:

Grid search can become expensive as the number of hyperparameters grows. RandomizedSearchCV explores a selected number of combinations more efficiently.

Q25. Why 5-fold CV?

Answer:

It provides a good balance between reliable validation and computational cost. Every observation is used for validation once.

Q26. Why not use accuracy?

Answer:

Accuracy is mainly used for classification. This is a regression problem, so MAE, RMSE, and R^2 are more appropriate.

Q27. MAE vs RMSE?

Answer:

MAE measures average absolute error, while RMSE penalizes large errors more heavily.

Q28. What does $R^2 = 0.919$ mean?

Answer:

It means the model explains approximately 91.9% of the variation in house prices on our held-out validation set.

Q29. Does $R^2 = 0.919$ mean 91.9% accuracy?

Answer:

No. R^2 is not accuracy. It measures explained variance for a regression model.

Q30. Why was stacking not chosen as the final model?

Answer:

After tuning, XGBoost achieved R^2 of about 0.919, while the final stacking ensemble achieved about 0.910. Since XGBoost performed better on the held-out validation set, I selected XGBoost based on empirical performance.

Q31. Why did feature engineering not become the final pipeline?

Answer:

I experimented with domain-driven features, but the feature-engineered XGBoost performed slightly worse than the baseline tuned version. Therefore, I kept the experiment but selected the simpler higher-performing pipeline.

Q32. Why test log transformation?

Answer:

SalePrice was right-skewed, so I tested log1p transformation to reduce skewness. However, it slightly reduced validation performance, so I did not use it in the final model.

Q33. What is SHAP?

Answer:

SHAP is an explainability method based on Shapley values that quantifies how individual features contribute to predictions.

Q34. Why SHAP?

Answer:

It helps explain both global feature importance and individual predictions, which is especially useful for a regression model that would otherwise be a black box.

Q35. What were the most important SHAP features?

Answer:

OverallQual, GrLivArea, TotalBsmSF, BsmFinSF1, GarageCars, YearBuilt and YearRemodAdd were among the strongest contributors.

Q36. What if a categorical value appears that wasn't in training?

Answer:

`handle_unknown="ignore"` prevents the encoder from failing and represents that unseen category appropriately.

Q37. How did you save the model?

Answer:

I used Joblib to serialize the trained model and preprocessing pipeline so they can be reloaded without retraining.

Q38. How would you predict a new house?

Answer:

```
model = joblib.load(  
    "final_house_price_model.joblib"  
)  
  
prediction = model.predict(  
    new_house  
)
```

The saved pipeline automatically handles preprocessing before prediction.

55. Tricky Cross Questions

Cross Question 1

Interviewer: "Your stacking model was worse than XGBoost. Why did you still implement stacking?"

Answer:

"Because the objective was to evaluate whether ensemble diversity improved generalization. I didn't assume stacking would win. I implemented proper OOF stacking, evaluated it, and selected XGBoost because it performed better."

Cross Question 2

"If XGBoost is already best, why include Random Forest and LightGBM?"

Answer:

"They provide different modeling approaches and error patterns. Comparing them gives us a stronger empirical basis for model selection and allows us to test whether ensemble methods can improve performance."

Cross Question 3

"Why shouldn't you train the meta-model using normal predictions from the training data?"

Answer:

"Those predictions are in-sample and overly optimistic because the base model has already seen the observations. OOF predictions provide predictions from models that did not train on those observations."

Cross Question 4

"Could your model be overfitting?"

Answer:

"There is always some overfitting risk, especially with tree ensembles. I controlled complexity through max depth, regularization, subsampling, hyperparameter tuning, and evaluation on held-out data."

Cross Question 5

"Why not remove all outliers?"

Answer:

"An unusual observation isn't necessarily an incorrect observation. I inspected extreme values and avoided deleting them blindly because removing valid expensive houses could reduce the model's ability to predict real high-value properties."

Cross Question 6

"Why didn't you standardize all numerical features?"

Answer:

"Our main models are tree-based, and tree splits are not sensitive to feature scale in the same way that distance-based or gradient-based linear models are. Therefore, standardization was not necessary for the final tree-based model."

56. Important Limitations

Be honest about these.

Limitation 1

The dataset has only 1460 labeled houses.

Limitation 2

The final performance is based on a held-out validation set, not a production housing market.

Limitation 3

The Kaggle test set does not expose true SalePrice locally.

Limitation 4

Feature engineering did not improve the final validation result.

Limitation 5

Stacking did not outperform tuned XGBoost.

Limitation 6

We did not deploy or monitor the model in production.

57. What I Would Improve in a Real Production System

If asked "What would you do next?", answer:

"I would validate the model on a larger and more recent housing dataset, perform stronger cross-validation, investigate temporal and geographic drift, calibrate uncertainty intervals, monitor prediction error after deployment, and retrain periodically as the housing market changes."

This is a very strong advanced answer.

58. 60-Second Project Explanation

Memorize this:

"I built an end-to-end house price prediction pipeline using the Kaggle House Prices dataset. I started with EDA to understand missing values, categorical variables, outliers, correlations and target skewness. I split the data into 80% training and 20% validation, then built a leakage-aware preprocessing pipeline using median imputation and one-hot encoding. I compared Random Forest, XGBoost and LightGBM. I also experimented with advanced feature engineering, target transformation and stacking using OOF predictions with a Ridge meta-model. After hyperparameter tuning, XGBoost achieved the best validation performance with an R^2 of about 0.919, MAE around \$15.7K and RMSE around \$24.9K. I used SHAP to explain feature importance and individual predictions and serialized the preprocessing plus model pipeline with Joblib for reproducible inference."

59. 20-Second Version

"I built a house price regression pipeline using Random Forest, XGBoost and LightGBM, with leakage-aware preprocessing, feature engineering, hyperparameter tuning and OOF stacking. Tuned XGBoost performed best with $R^2 \approx 0.919$, and I used SHAP for explainability."

60. Final Cheat Sheet

Problem:
Supervised regression

Dataset:
Kaggle House Prices
1460 rows
80 input features
SalePrice target

Split:
80% train
20% validation

Preprocessing:
Median imputation
One-hot encoding
handle_unknown="ignore"

Models:

Random Forest
XGBoost
LightGBM

Final Model:
Tuned XGBoost

Best Parameters:
n_estimators=900
learning_rate=0.02
max_depth=3
min_child_weight=3
subsample=0.9
colsample_bytree=0.8
reg_alpha=0.01
reg_lambda=1

Stacking:
RF + XGB + LGB
OOF predictions
Ridge meta-model
alpha=1.0

Result:
Stacking < Tuned XGBoost

Final Validation:
 $R^2 \approx 0.919$
RMSE $\approx$ \$24.9K
MAE $\approx$ \$15.7K

Explainability:
SHAP

Saving:
Joblib

Deployment:
Not deployed

61. Three Things You MUST Remember in the Interview

1. Why XGBoost?

Strong tabular performance and best result in our experiments.

2. Why OOF stacking?

To generate honest predictions for the meta-model and prevent leakage.

3. Why XGBoost instead of stacking?

Because the tuned XGBoost performed better on the held-out validation set.

62. Most Important Rule

Never claim:

"My model is 90.82% accurate."

Say:

"My tuned XGBoost achieved $R^2 \approx 0.919$, RMSE $\approx$ \$24.9K, and MAE $\approx$ \$15.7K on the held-out validation set."

That is technically correct and interview-safe.